

```

        memcpy(buffer + size, msg, msgLen);
        size += msgLen;
        buffer[size] = '\0';
    } else {
        cout << "[Warning] Not enough capacity to append message." << endl;
    }
}

void print() const {
    accessCount++;
    cout << buffer << endl;
}

void clear() {
    size = 0;
    buffer[0] = '\0';
}

static int getInstanceCount() {
    return instanceCount;
}

int getAccessCount() const {
    return accessCount;
}
};

int LogBuffer::instanceCount = 0;

int main() {
    LogBuffer log1(256);
    log1.append("Server started on port 8080");
    log1.append(" | Request received from 192.168.1.10");
    log1.print();
    cout << endl;
    LogBuffer log2 = log1;
    log2.append(" | Cached response sent");
    cout << "log1 : "; log1.print();
    cout << "log2 : "; log2.print();
    cout << endl;
    LogBuffer log3(128);
    log3 = log1;
    log3.print();
    cout << endl;
    log1 = log1;
    log1.print();
    cout << endl;
    cout << "Live LogBuffer objects : " << LogBuffer::getInstanceCount() << endl;
    cout << endl;
    const LogBuffer readOnly(log1);
    readOnly.print();
    readOnly.print();
    readOnly.print();
    cout << "Times printed: " << readOnly.getAccessCount() << endl;
}

```

```

#include <iostream>
#include <cstring>
using namespace std;
class LogBuffer {
private:
    char* buffer;
    int capacity;
    int size;
    static int instanceCount;
    mutable int accessCount = 0;
public:
    LogBuffer(int cap) {
        capacity = cap;
        size = 0;
        buffer = new char[capacity];
        buffer[0] = '\0';
        instanceCount++;
        cout << "[LogBuffer Created] capacity=" << capacity << endl;
    }
    LogBuffer(const LogBuffer& other) {
        capacity = other.capacity;
        size = other.size;
        buffer = new char[capacity];
        memcpy(buffer, other.buffer, size + 1);
        instanceCount++;
        cout << "[LogBuffer Deep Copied] capacity=" << capacity << endl;
    }
    LogBuffer& operator=(const LogBuffer& other) {
        if (this == &other) {
            cout << "[Self-assignment detected - no operation]" << endl;
            return *this;
        }
        delete[] buffer;
        capacity = other.capacity;
        size = other.size;
        buffer = new char[capacity];
        memcpy(buffer, other.buffer, size + 1);
        cout << "[LogBuffer Assigned]" << endl;
        return *this;
    }
    ~LogBuffer() {
        delete[] buffer;
        instanceCount--;
        cout << "[LogBuffer Destroyed]" << endl;
    }
    void append(const char* msg) {
        int msgLen = strlen(msg);
        if (size + msgLen < capacity) {

```

```
[LogBuffer Created] capacity=256
Server started on port 8080 | Request received from 192.168.1.10

[LogBuffer Deep Copied] capacity=256
log1 : Server started on port 8080 | Request received from 192.168.1.10
log2 : Server started on port 8080 | Request received from 192.168.1.10 | Cached response sent

[LogBuffer Created] capacity=128
[LogBuffer Assigned]
Server started on port 8080 | Request received from 192.168.1.10

[Self-assignment detected - no operation]
Server started on port 8080 | Request received from 192.168.1.10

Live LogBuffer objects : 3

[LogBuffer Deep Copied] capacity=256
Server started on port 8080 | Request received from 192.168.1.10
Server started on port 8080 | Request received from 192.168.1.10
Server started on port 8080 | Request received from 192.168.1.10
Times printed: 3

[LogBuffer Destroyed]
[LogBuffer Destroyed]
[LogBuffer Destroyed]
[LogBuffer Destroyed]
```

```

private:
    string name;
    float duration;
public:
    AudioClip(string n, float d) {
        name = n;
        duration = d;
        cout << "[AudioClip Loaded] " << name << endl;
    }
    ~AudioClip() {
        cout << "[AudioClip Released] " << name << endl;
    }
    string getName() {
        return name;
    }
};

int main() {
    cout << "==== PART A: unique_ptr =====> << endl;
    auto tex1 = make_unique<Texture>("player_sprite", 512, 512);
    tex1->display();
    unique_ptr<Texture> tex2 = move(tex1);
    cout << "tex1 is null: " << (tex1 == nullptr ? "YES" : "NO") << endl;
    cout << "\n==== PART B: shared_ptr =====> << endl;
    auto shader = make_shared<Shader>("main_vert", "vertex");
    cout << "Ref count: " << shader.use_count() << endl;
    {
        cout << "Ref count: " << shader.use_count() << endl;
        {
            auto editorRef = shader;
            cout << "Ref count: " << shader.use_count() << endl;
        }
        cout << "Ref count: " << shader.use_count() << endl;
    }
    cout << "Ref count: " << shader.use_count() << endl;
    cout << "\n==== PART C: weak_ptr =====> << endl;
    auto audio = make_shared<AudioClip>("explosion", 3.5f);
    weak_ptr<AudioClip> observer = audio;
    if (auto clip = observer.lock()) {
        cout << "Clip alive: " << clip->getName() << endl;
    }
    audio.reset();
    if (observer.expired()) {
        cout << "Clip already unloaded." << endl;
    }
}

```

```

#include <iostream>
#include <string>
#include <memory>
using namespace std;
class Texture {
private:
    string name;
    int width;
    int height;
public:
    Texture(string n, int w, int h) {
        name = n;
        width = w;
        height = h;
        cout << "[Texture Loaded] " << name << endl;
    }
    ~Texture() {
        cout << "[Texture Released] " << name << endl;
    }
    void display() const {
        cout << "Texture: " << name << " (" << width << "x" << height << ")" << endl;
    }
};

class Shader {
private:
    string name;
    string type;
public:
    Shader(string n, string t) {
        name = n;
        type = t;
        cout << "[Shader Compiled] " << name << " (" << type << ")" << endl;
    }
    ~Shader() {
        cout << "[Shader Destroyed] " << name << endl;
    }
    int getReferenceCount(shared_ptr<Shader> s) {
        return s.use_count();
    }
};

class AudioClip {
private:
    string name;

```

```
===== PART A: unique_ptr =====  
[Texture Loaded] player_sprite  
Texture: player_sprite (512x512)  
tex1 is null: YES
```

```
===== PART B: shared_ptr =====  
[Shader Compiled] main_vert (vertex)  
Ref count: 1  
Ref count: 1  
Ref count: 2  
Ref count: 1  
Ref count: 1
```

```
===== PART C: weak_ptr =====  
[AudioClip Loaded] explosion  
Clip alive: explosion  
[AudioClip Released] explosion  
Clip already unloaded.  
[Shader Destroyed] main_vert  
[Texture Released] player_sprite
```

```

        default:
            cout << endpoint << " -> Unknown status code" << endl;
            break;
    }
}

void printHeaders(const HeaderList& headers) {
    cout << "Headers:" << endl;
    for (int i = 0; i < headers.size(); i++) {
        cout << " " << headers[i].first << " : " << headers[i].second << endl;
    }
}

int main() {
    cout << "----- Part A: Handling HTTP Responses -----" << endl;
    handleResponse(HttpStatus::OK, "[GET /api/users]");
    handleResponse(HttpStatus::Unauthorized, "[POST /api/login]");
    handleResponse(HttpStatus::NotFound, "[GET /api/products/99]");
    handleResponse(HttpStatus::ServerError, "[POST /api/order]");
    cout << endl;
    cout << "----- Part B: Type Aliases -----" << endl;
    RequestId reqId = 1748293847;
    Port serverPort = 8080;
    cout << "Request ID   : " << reqId << endl;
    cout << "Server Port   : " << serverPort << endl;
    cout << endl;
    HeaderList headers;
    headers.push_back(make_pair("Content-Type", "application/json"));
    headers.push_back(make_pair("Authorization", "Bearer eyJhbGciOi.."));
    headers.push_back(make_pair("Accept-Language", "en-US"));
    printHeaders(headers);
    return 0;
}

```



```

#include <iostream>
#include <string>
#include <vector>
#include <utility>
using namespace std;
enum class HttpStatus {
    OK = 200,
    Created = 201,
    BadRequest = 400,
    Unauthorized = 401,
    NotFound = 404,
    ServerError = 500
};
using Header = pair<string, string>;
using HeaderList = vector<Header>;
using Port = unsigned int;
using IPAddress = string;
typedef unsigned long long RequestId;
void handleResponse(HttpStatus status, const string& endpoint) {
    switch (static_cast<int>(status)) {
        case static_cast<int>(HttpStatus::OK):
            cout << endpoint << " -> 200 OK : Request successful" << endl;
            break;
        case static_cast<int>(HttpStatus::Created):
            cout << endpoint << " -> 201 Created : Resource created successfully" << endl;
            break;
        case static_cast<int>(HttpStatus::BadRequest):
            cout << endpoint << " -> 400 Bad Request : Invalid request syntax" << endl;
            break;
        case static_cast<int>(HttpStatus::Unauthorized):
            cout << endpoint << " -> 401 Unauthorized : Authentication required" << endl;
            break;
        case static_cast<int>(HttpStatus::NotFound):
            cout << endpoint << " -> 404 Not Found : Endpoint does not exist" << endl;
            break;
        case static_cast<int>(HttpStatus::ServerError):
            cout << endpoint << " -> 500 Server Error : Internal server error - retry later" << endl;
            break;
        default:
            cout << endpoint << " -> Unknown status code" << endl;
            break;
    }
}

```


terminated (exit value 0) Day 11: Node [C] > C:\Program Files (x86)\Node.js\node [04/03/2019]

```
----- Part A: Handling HTTP Responses -----  
[GET /api/users] -> 200 OK : Request successful  
[POST /api/login] -> 401 Unauthorized : Authentication required  
[GET /api/products/99] -> 404 Not Found : Endpoint does not exist  
[POST /api/order] -> 500 Server Error : Internal server error - retry later
```

```
----- Part B: Type Aliases -----
```

```
Request ID    : 1748293847
```

```
Server Port   : 8080
```

```
Headers:
```

```
Content-Type  : application/json
```

```
Authorization : Bearer eyJhbGci...
```

```
Accept-Language : en-US
```